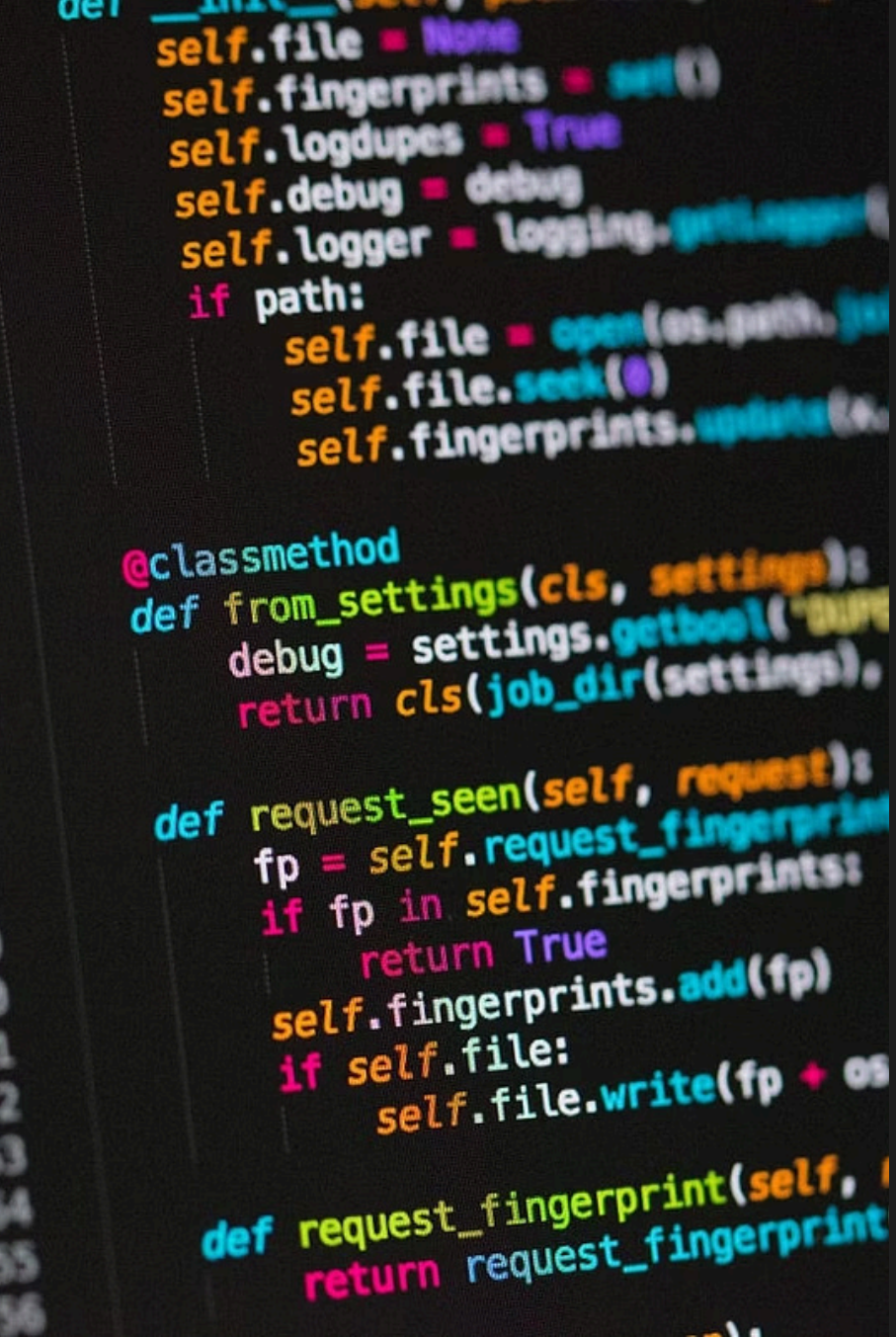


Python for Data Science

Loops, Functions, Modules, and Packages — A Beginner's Journey

Made with GAMMA



What Are Loops and Why Do We Need Them?

The Repetition Problem

Imagine you need to greet 100 students in a class. Writing `print("Hello!")` 100 times would be exhausting! Loops solve this by automating repetition.

Real-World Examples

- Scrolling through your Instagram feed
- Processing each item in a shopping cart
- Checking every student's grade in a list
- Reading through all emails in your inbox

Made with GAMMA

The for Loop: Your First Automation Tool

01

Understanding the Syntax

`for item in sequence:` means "do something with each item in this collection"

02

Common Sequences

Lists, strings, tuples — any collection of items you want to process one by one

03

Indentation Matters

Python uses spacing to know which code belongs inside the loop — always indent by 4 spaces

Common Beginner Mistake

Forgetting the colon `:` at the end of the first line or mixing up indentation levels

Made with GAMMA

for Loop in Action

Example 1: Greeting Students

```
students = ["Alice", "Bob", "Charlie"]
for student in students:
    print(f"Hello, {student}!")
```

Output:

```
Hello, Alice!
Hello, Bob!
Hello, Charlie!
```

Example 2: Processing Letters

```
message = "Python"
for letter in message:
    print(letter.upper())
```

Output:

```
P
Y
T
H
O
N
```



Made with GAMMA

The range() Function: Creating Number Sequences

range(stop)

Numbers from 0 to stop-1

```
range(5)
# 0, 1, 2, 3, 4
```

range(start, stop)

Numbers from start to stop-1

```
range(2, 6)
# 2, 3, 4, 5
```

range(start, stop, step)

Numbers with custom intervals

```
range(1, 10, 2)
# 1, 3, 5, 7, 9
```

Remember: range() doesn't include the stop value — it goes up to but not including it!

Made with GAMMA

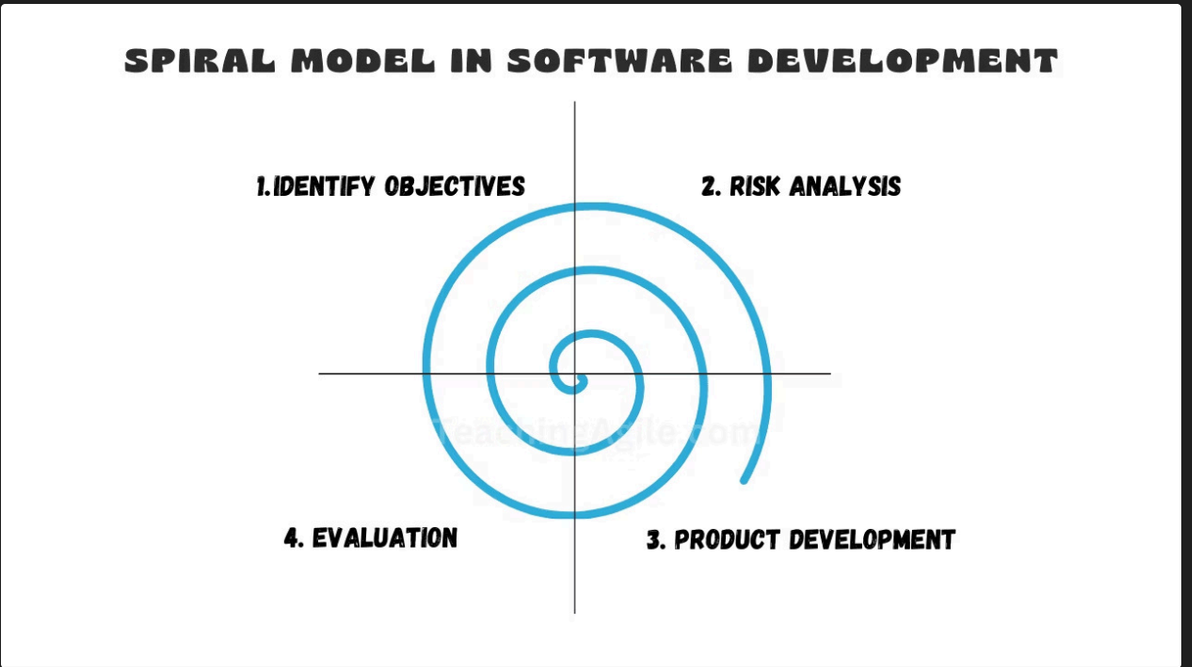
while Loop: Looping with Conditions

When to Use while

Use `while` when you don't know exactly how many times to loop — it continues as long as a condition is `True`.

Watch Out for Infinite Loops!

Always ensure your condition will eventually become `False`, or your program will run forever.



Basic Syntax

```
while condition:
    # code block
    # update condition
```

Counter Example

```
count = 0
while count < 3:
 print(count)
 count += 1
# 0, 1, 2
```

Made with GAMMA

Loop Control Statements

break

Exits the loop immediately, regardless of the condition. Like finding what you're looking for and stopping the search.

continue

Skips the current iteration and moves to the next one. Like skipping over items that don't meet your criteria.

pass

Does nothing — a placeholder when syntax requires code but you're not ready to write it yet.



Made with GAMMA

Nested Loops: Loops Inside Loops

The Concept

Think of a grid: the outer loop handles rows, the inner loop handles columns. For each row, all columns are processed.

Visual Pattern

```
for i in range(3):
    for j in range(2):
        print(f"Row {i}, Col {j}")
```

Example Output

```
Row 0, Col 0
Row 0, Col 1
Row 1, Col 0
Row 1, Col 1
Row 2, Col 0
Row 2, Col 1
```

⚠ **Performance Note**

Nested loops can be slow with large datasets. Use them carefully and consider if there's a more efficient approach!

Made with GAMMA

Loop with else Clause

1

Normal Loop Completion

If the loop finishes naturally (without `break`), the `else` block runs

2

Loop with break

If `break` is triggered, the `else` block is skipped

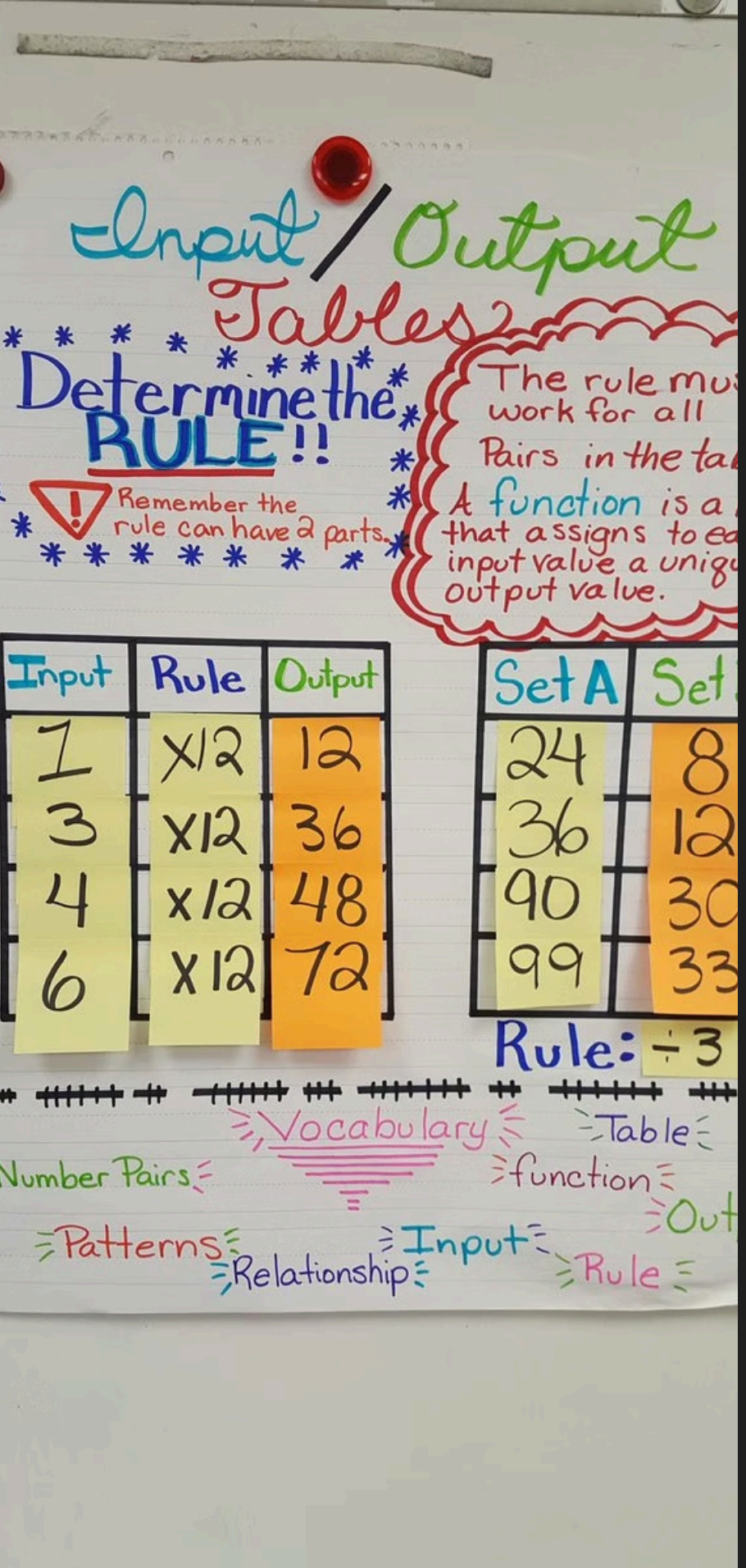
Example: Finding a Number

```
numbers = [1, 2, 3, 4, 5]
target = 6

for num in numbers:
    if num == target:
        print(f"Found {target}!")
        break
else:
    print(f"{target} not found in list")
```

Output: `6 not found in list`

Made with GAMMA



Functions: Reusable Code Blocks

Why Functions?

Write code once, use it many times. Keep your programs organized and avoid repeating yourself.

def Keyword

Start every function with `def function_name():` followed by indented code

Calling Functions

Use `function_name()` to execute the code inside the function

Simple Function Example

```
def greet():
 print("Hello, welcome to Python!")

greet() # Call the function
# Output: Hello, welcome to Python!
```

Made with GAMMA

Functions: Parameters and Arguments

To make functions flexible and perform different actions based on input, we use parameters and arguments.

Parameters: The Blueprint

Variables listed inside the parentheses in the function's definition. They act as placeholders for values the function needs to operate.

Arguments: The Actual Values

The actual values passed to the function when it is called. These values are assigned to the corresponding parameters.

Positional Arguments

The simplest way to pass arguments is by their position. Python matches arguments to parameters in the order they are provided.

Example: Greeting with a Name

```
def greet(name, time_of_day):
    print(f"Good {time_of_day}, {name}!")

greet("Alice", "morning")
greet("Bob", "evening")
```

Output & Explanation

```
Good morning, Alice!
Good evening, Bob!
```

Here, `"Alice"` and `"Bob"` are positional arguments for the `name` parameter, and `"morning"` and `"evening"` for `time_of_day`.

Analogy: A Recipe

Think of a **recipe** as a function. The **ingredients list** (like "sugar", "flour") are its **parameters**. When you actually **bake a cake**, the **specific amounts** you add (e.g., "1 cup of sugar", "2 cups of flour") are the **arguments**.

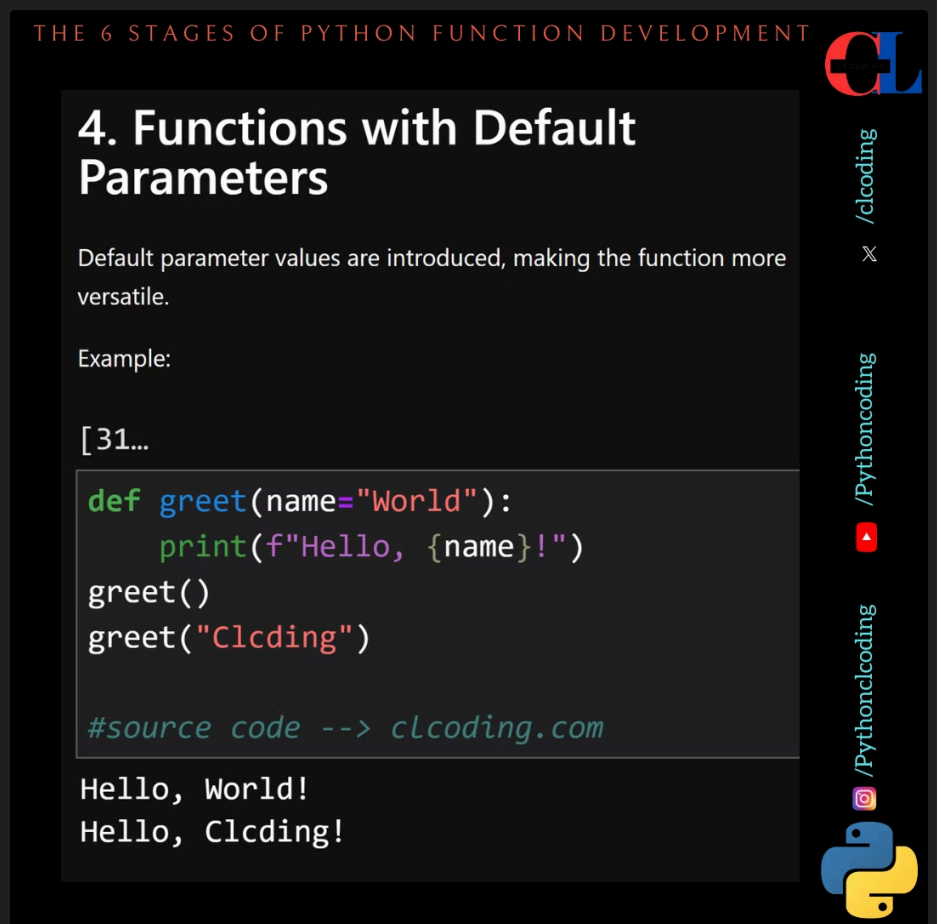
Made with GAMMA

Default Arguments

Default arguments allow you to assign a default value to a function parameter. If the caller doesn't provide a value for that parameter, the default value is used. This makes functions more flexible and prevents errors when certain arguments are optional.

Why They Are Useful

- **Flexibility:** Call the function with fewer arguments for common use cases.
- **Readability:** Clearly indicates optional parameters and their standard values.
- **Maintainability:** Avoids needing to write multiple functions for slightly different behaviors.



Example: Customizable Greeting

```
def greet(name, message="Hello"):
    print(f"{message}, {name}!")

greet("Alice")       # Uses default message
greet("Bob", "Hi")    # Overrides default message
greet(message="Good morning", name="Charlie")
# Keyword args also work
```

Output:

```
Hello, Alice!
Hi, Bob!
Good morning, Charlie!
```

Made with GAMMA

Keyword Arguments

Keyword arguments allow you to pass values to a function by explicitly naming the parameter they correspond to, rather than relying solely on their position.

Enhanced Readability

Function calls become self-documenting, making the code easier to understand at a glance, especially for functions with many parameters.

Order Independence

You can provide arguments in any order, as long as you specify the parameter name, which increases flexibility.

Preventing Errors

Reduces the chance of passing values to the wrong parameters, especially when some parameters have default values.

Example: Customizing a Report

```
def create_report(title, author="Anonymous", format="PDF"):
    print(f"Generating Report:")
    print(f"Title: {title}")
    print(f"Author: {author}")
    print(f"Format: {format}")

# Using keyword arguments
create_report(title="Quarterly Review", format="DOCX", author="Dr. Smith")
print("\n---")
create_report(format="TXT", title="Daily Log") # Order doesn't matter
```

Output:

```
Generating Report:
Title: Quarterly Review
Author: Dr. Smith
Format: DOCX

---
Generating Report:
Title: Daily Log
Author: Anonymous
Format: TXT
```

Notice how the order of `format` and `title` doesn't matter in the second call, and the default `author` is used because it wasn't specified.

Made with GAMMA

*args and **kwargs: Flexible Function Inputs

Python provides special syntax to allow functions to accept a variable number of positional arguments (`*args`) and keyword arguments (`**kwargs`). This makes your functions incredibly flexible and reusable.

1

*args (Arbitrary Positional Arguments)

The single asterisk `*` gathers all extra positional arguments into a **tuple**. You can then iterate over this tuple inside your function.

```
def sum_all(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_all(1, 2, 3, 4))
# Output: 10
```

2

**kwargs (Arbitrary Keyword Arguments)

The double asterisk `**` collects all extra keyword arguments into a **dictionary**. This allows functions to accept an undefined number of named parameters.

```
def build_profile(**details):
    for key, value in details.items():
        print(f"{key.replace('_', ' ').title()}: {value}")

build_profile(name="Alice", age=30, city="NY")
# Output:
# Name: Alice
# Age: 30
# City: NY
```

Analogy: Ordering at a Restaurant 🍽️

Imagine ordering food:

- **`*args`**: You tell the waiter your main dishes: "I'll have the **pizza, pasta, and salad**." (A list of items, in order).
- **`**kwargs`**: You then add special requests or modifications: "And for the pizza, I want **extra_cheese='yes'** and **no_olives='true'**." (Named, optional additions).

Combining *args and **kwargs

You can use both in the same function definition. `*args` must come before `**kwargs`.

```
def configure_system(main_setting, *options,
**settings):
    print(f"Main Setting: {main_setting}")
    if options:
        print(f"Options: {options}") # Tuple
    if settings:
        print(f"Custom Settings: {settings}") # Dictionary

configure_system(
    "startup",
    "verbose", "debug_mode", # *options
    log_level="INFO",
    max_retries=5          # **settings
)
```

Output:

```
Main Setting: startup
Options: ('verbose', 'debug_mode')
Custom Settings: {'log_level': 'INFO', 'max_retries': 5}
```

This demonstrates how `*args` captures the two string options as a tuple, and `**kwargs` captures the `log_level` and `max_retries` as a dictionary of key-value pairs.

Made with GAMMA

Return Statements

The `return` statement is how a Python function sends a value or result back to the part of the code that called it. This is fundamental for building functions that perform computations and pass their outcomes for further processing.

`print()`: For Displaying

Displays output to the console for human viewing. It does not pass any value back to the caller. Its primary purpose is side effects (showing information).

```
def show_message(msg):
    print(msg) # Displays "Hello"
show_message("Hello")
result = show_message("Hello")
print(result) # Output: None
```

`return`: For Providing a Value

Sends a value back to the caller and immediately exits the function. This value can be stored in a variable or used in expressions. Its primary purpose is computation and data transfer.

```
def get_message(msg):
    return msg # Returns "Hello"
message = get_message("Hello")
print(message) # Output: Hello
```

Function Output Flow

When Python encounters a `return` statement, the function execution stops, and the specified value is immediately sent back to the calling context. Any code after `return` in the function will not execute.

```
def calculate_area(length, width):
    area = length * width
    return area # Sends 'area' back

def get_status(score):
    if score >= 60:
        return "Pass"
    else:
        return "Fail"
    print("This won't run!") # Unreachable

# Using the returned values
room_area = calculate_area(5, 10)
print(f"Room area: {room_area} sq. units")

student_result = get_status(75)
print(f"Student status: {student_result}")

another_student_result = get_status(45)
print(f"Another student status:
{another_student_result}")
```

Explanation & Output:

The `calculate_area` function computes the area and then explicitly returns that numerical value. This value is then stored in the `room_area` variable for later use.

The `get_status` function returns a string based on a condition. Notice how the `print("This won't run!")` line is never executed because `return` immediately exits the function.

```
Room area: 50 sq. units
Student status: Pass
Another student status: Fail
```

Made with GAMMA

Variable Scope: Local vs. Global

In Python, **variable scope** defines where a variable can be accessed, created, or modified within your program. Understanding scope prevents unexpected errors and makes your code more predictable.

Local Variables

Variables defined **inside a function** are local to that function. They can only be accessed from within that function's block. Once the function completes, these variables are destroyed.

```
def greet():
    message = "Hello from local" # Local variable
    print(message)

greet()
# print(message) # This would cause an error (NameError)
```

Global Variables

Variables defined **outside of any function** (at the top level of a script or module) are global. They can be accessed from anywhere in the program, including inside functions. To modify a global variable inside a function, you need to explicitly declare it using the `global` keyword.

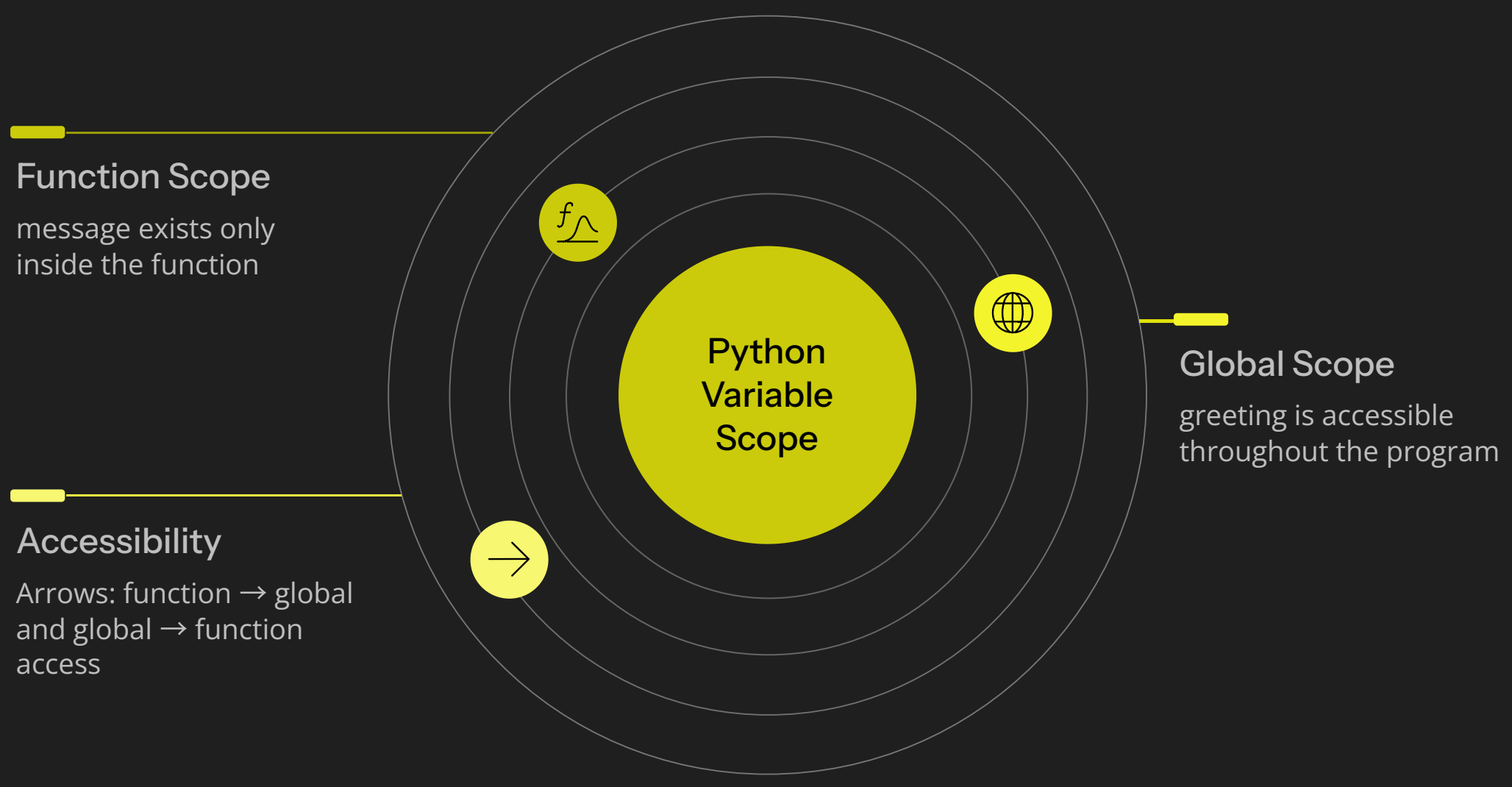
```
greeting = "Hello from global" # Global variable

def show_greeting():
    print(greeting) # Access global variable

def change_greeting():
    global greeting # Declare intent to modify global
    greeting = "Updated global greeting"

show_greeting()
change_greeting()
show_greeting()
```

Visualizing Variable Scope



This diagram shows how global variables exist at the top level of your program and are generally accessible everywhere, while local variables are confined to the specific function where they are defined, created when the function is called, and destroyed when it exits.

⚠ Common Mistakes

- Attempting to access a local variable outside its function scope.
- Forgetting to use the `global` keyword when trying to modify a global variable inside a function, which often leads to Python creating a new local variable with the same name instead.

Made with GAMMA

Lambda Functions

Lambda functions are small, anonymous functions defined using the `lambda` keyword. Unlike traditional functions (`def`), they don't have a name and are restricted to a single expression. They are primarily used for quick, inline operations where a full function definition would be verbose.

Syntax & Use Cases

```
# Syntax: lambda arguments: expression

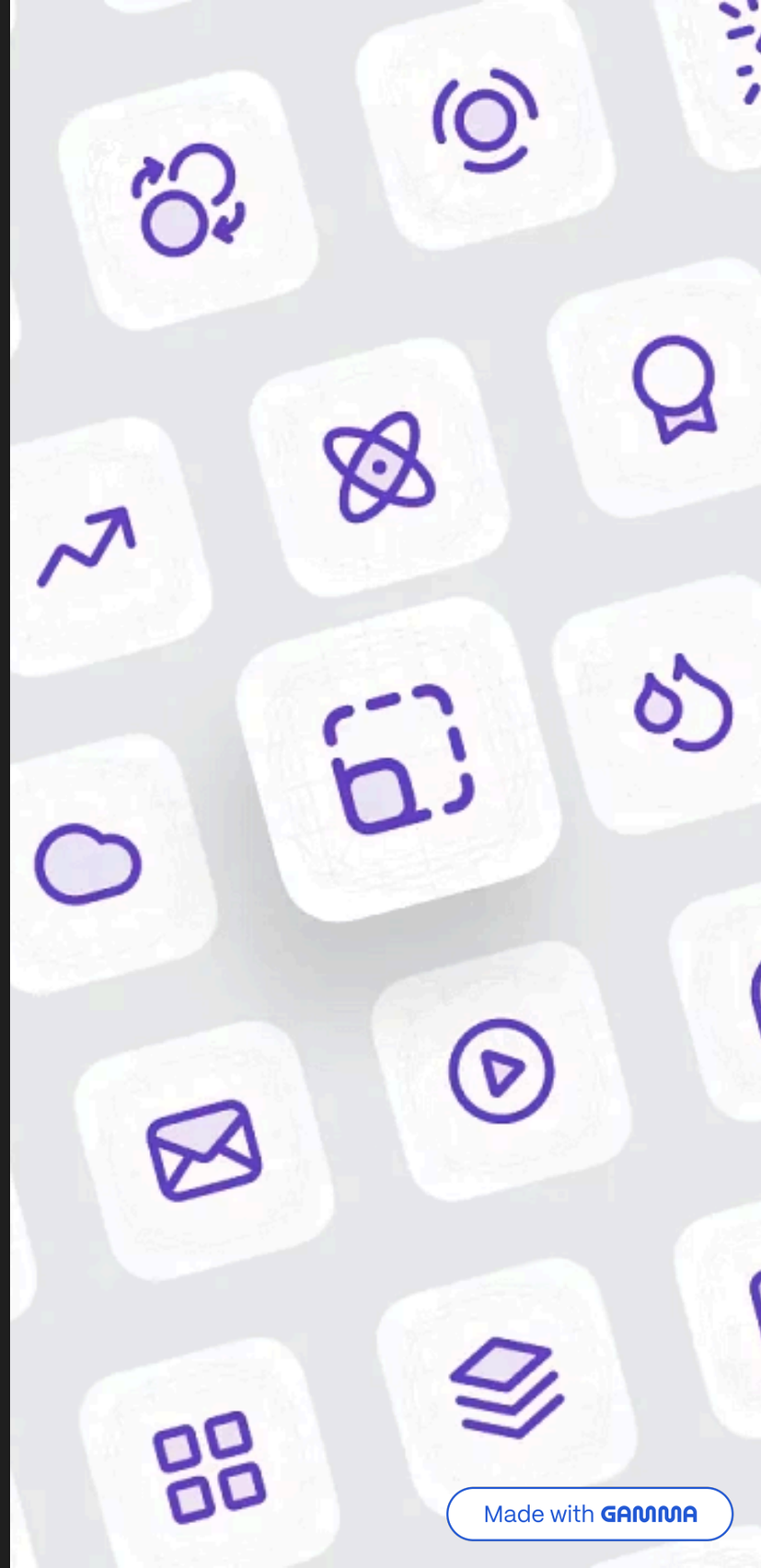
# Example 1: Adding two numbers
add = lambda a, b: a + b
print(add(3, 5)) # Output: 8

# Example 2: Squaring a number
square = lambda x: x * x
print(square(7)) # Output: 49
```

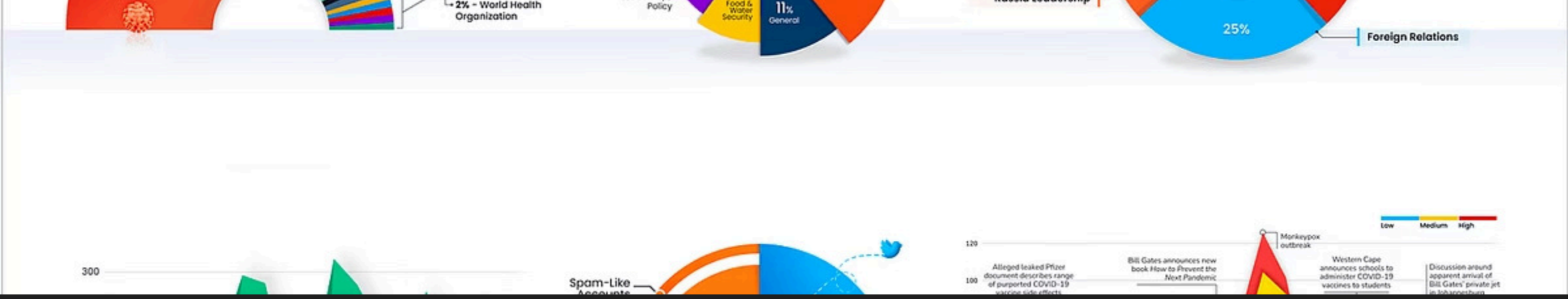
Lambda functions shine in situations requiring a small function for a short period, especially as arguments to higher-order functions like `map()`, `filter()`, or `sort()`'s `key` parameter. They offer a concise alternative to defining a full function with `def`.

Key Difference

`def` creates a named function with multiple statements; `lambda` creates an anonymous function with a single expression.



Made with GAMMA



Higher-Order Functions: `map()`, `filter()`, `zip()`

Python offers powerful built-in higher-order functions like `map()`, `filter()`, and `zip()` that simplify data manipulation by processing iterables efficiently. These functions are cornerstones in data science for transforming, selecting, and combining datasets without explicit loops, leading to more concise and readable code.

`map()`: Apply a Function

Applies a given function to each item of an iterable (like a list) and returns a `map` object, which is an iterator. In data science, this is invaluable for feature engineering (e.g., scaling numerical features) or data cleaning (e.g., converting data types across an entire column).

```
# Double each number in a list
numbers = [1, 2, 3, 4]
doubled = list(map(lambda x: x * 2, numbers))
# Output: [2, 4, 6, 8]
```

Visually, `map()` takes a stream of input values, passes each one through a processing unit (the function), and generates a new stream of transformed output values. It's a one-to-one transformation for every element.

`filter()`: Select Elements

Constructs an iterator from elements of an iterable for which a function returns true. This is perfect for selecting specific rows based on criteria or cleaning outliers from datasets. It helps in focusing on relevant data subsets for analysis.

```
# Keep only even numbers
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Output: [2, 4, 6]
```

Imagine `filter()` as a sieve or a funnel: it lets some elements pass through based on a condition and discards others. It's a selective process that reduces the volume of data based on defined rules.

`zip()`: Combine Iterables

Combines elements from multiple iterables into tuples, pairing corresponding items. It stops when the shortest iterable is exhausted. This is extremely useful for creating temporary associations between different data arrays, like pairing features with labels, or combining two columns into key-value pairs.

```
# Combine names and scores
names = ["Alice", "Bob"]
scores = [90, 85]
paired_data = list(zip(names, scores))
# Output: [('Alice', 90), ('Bob', 85)]
```

`zip()` works like a zipper, taking elements from parallel sequences and merging them together, element by element, into a single new sequence of pairs or tuples. It synchronizes multiple lists into a unified structure.

Made with GAMMA

Modules in Python: Building Blocks for Robust Code

As your Python projects grow, keeping all your code in a single file becomes unmanageable. Python modules provide an elegant solution by allowing you to organize related code into separate files. Think of modules as toolboxes, each containing a specific set of tools (functions, classes, variables) that you can use across different parts of your application.

What Are Modules?

A module is simply a Python file (`.py`) containing Python definitions and statements. Any Python file can be a module. For instance, if you have a file named `my_math.py`, it can serve as a module.

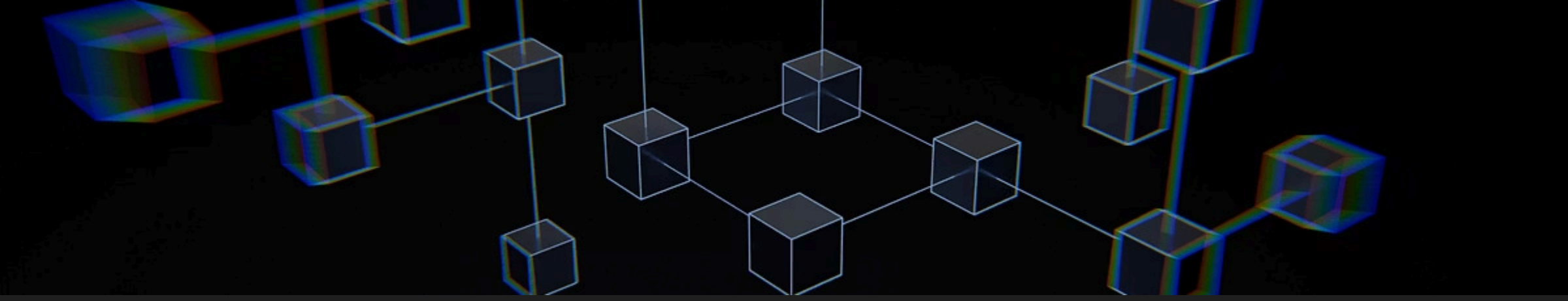
Why Are Modules Useful?

- **Reusability:** Use code written once in many programs.
- **Organization:** Group related code, making your project easier to understand and navigate.
- **Namespace Isolation:** Prevent naming conflicts between different parts of your code.
- **Collaboration:** Multiple developers can work on different modules simultaneously.

The Essence of Code Organization

Modules are fundamental to structuring larger Python applications, from simple scripts to complex data science pipelines. They enable a modular programming approach, breaking down complex problems into smaller, manageable, and interconnected components.





Built-in Modules: Expanding Python's Capabilities

Python's strength lies not only in its core language but also in its extensive collection of built-in modules. These pre-written code libraries offer ready-to-use functionalities, saving development time and effort. Let's explore some fundamental ones.

`math` Module

Provides access to mathematical functions for both real and complex numbers. Essential for scientific and data computations.

```
import math
print(math.sqrt(25)) # 5.0
print(math.ceil(4.2))  # 5
print(math.floor(4.9)) # 4
print(math.pi)       # 3.14159...
```

`os` Module

Offers a way to interact with the operating system, enabling file and directory operations like navigating paths or listing contents.

```
import os
print(os.getcwd()) # Current directory
# os.mkdir("new_folder")
# os.remove("file.txt")
```

`sys` Module

Provides access to system-specific parameters and functions, useful for command-line arguments and interacting with the interpreter.

```
import sys
# print(sys.argv) # Cmd line args
print(sys.version) # Python version
# sys.exit()
```

Made with GAMMA

The `random` Module: Embracing Chance in Data

The `random` module in Python is a fundamental tool for generating pseudorandom numbers and sequences. It's essential in data science for tasks ranging from simulating experiments and creating random samples to shuffling datasets for unbiased model training. Understanding its functions allows you to introduce controlled randomness into your analyses.

`random()`: Float [0.0, 1.0)

Generates a random floating-point number between 0.0 (inclusive) and 1.0 (exclusive). Ideal for probability simulations and normalized random values.

```
import random
print(random.random()) # e.g., 0.835
```

`randint(a, b)`: Integer Range

Returns a random integer *N* such that *a* <= *N* <= *b*. Perfect for selecting elements by index or simulating discrete events.

```
import random
print(random.randint(1, 10)) # e.g., 5
```

`choice(seq)`: Random Element

Returns a random element from a non-empty sequence (e.g., list, tuple, string). Useful for random sampling or decision-making processes.

```
import random
data = ["A", "B", "C"]
print(random.choice(data)) # e.g., "B"
```

`shuffle(list)`: In-place Shuffle

Randomly shuffles the elements of a sequence *in place*, meaning it modifies the original list. Crucial for randomizing datasets before training machine learning models.

```
import random
cards = [1, 2, 3, 4]
random.shuffle(cards)
print(cards) # e.g., [3, 1, 4, 2]
```

Real-world Examples

The `random` module's versatility makes it indispensable in various data science applications:

- **Games and Simulations:** Generating dice rolls, card draws, or random events in complex system simulations.
- **Statistical Sampling:** Creating random subsets of large datasets for faster analysis or cross-validation in machine learning.
- **Security and Cryptography:** Though Python's `secrets` module is preferred for cryptographically secure randomness, `random` can be used for less sensitive applications like generating temporary tokens.

Made with GAMMA



The `datetime` Module: Mastering Time in Data Science

Handling dates and times is a common yet critical task in data analysis, machine learning, and reporting. Python's built-in `datetime` module provides classes to manage dates, times, and time intervals with precision and flexibility. It's indispensable for timestamping events, calculating durations, and performing time-series analysis.

`date` Object

Represents a calendar date (year, month, day). Ideal for managing dates without time components.

```
from datetime import date
today = date.today()
print(f"Today's date: {today}")
# Output: Today's date: 2023-10-27 (example)
```

`time` Object

Represents a local time of day (hour, minute, second, microsecond). Useful when only the time is needed.

```
from datetime import time
lunch_time = time(12, 30, 0)
print(f"Lunch: {lunch_time}")
# Output: Lunch: 12:30:00
```

`datetime` Object

Combines both date and time into a single object. This is the most frequently used class for complete timestamp information.

```
from datetime import datetime
now = datetime.now()
print(f"Current datetime: {now}")
# Output: Current datetime: 2023-10-27 10:45:00.123456 (example)
```

`timedelta` Object

Represents a duration, the difference between two `date`, `time`, or `datetime` instances. Essential for calculations involving time intervals.

```
from datetime import datetime, timedelta
future_date = datetime.now() + timedelta(days=7, hours=3)
print(f"In 1 week and 3 hours: {future_date}")
# Output: In 1 week and 3 hours: 2023-11-03 13:45:00.123456 (example)
```

The `datetime` module simplifies complex time operations, allowing data professionals to clean, transform, and analyze time-based data effectively.

Made with GAMMA

pip – Python Package Manager

pip is the essential tool for working with Python's rich ecosystem of external libraries. It simplifies the process of installing, managing, and updating software packages, enabling data scientists to leverage powerful tools developed by the community.

What is pip?

Python's standard package-management system. It enables installation and management of software packages from the Python Package Index (PyPI).

Why Packages Are Needed

Packages extend Python's capabilities, providing pre-written, tested code for specific tasks, saving development time and effort.

Installing Libraries

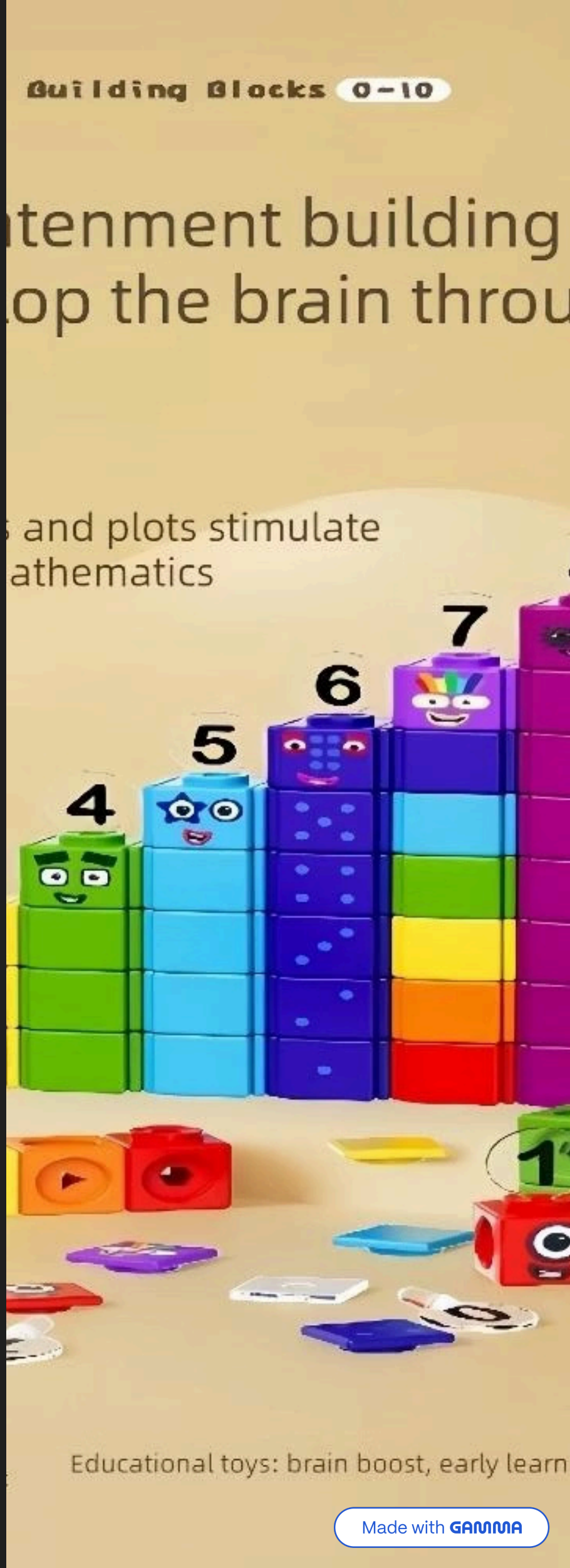
Easily install external libraries using `pip install package_name` in your terminal. `pip` handles dependencies automatically.

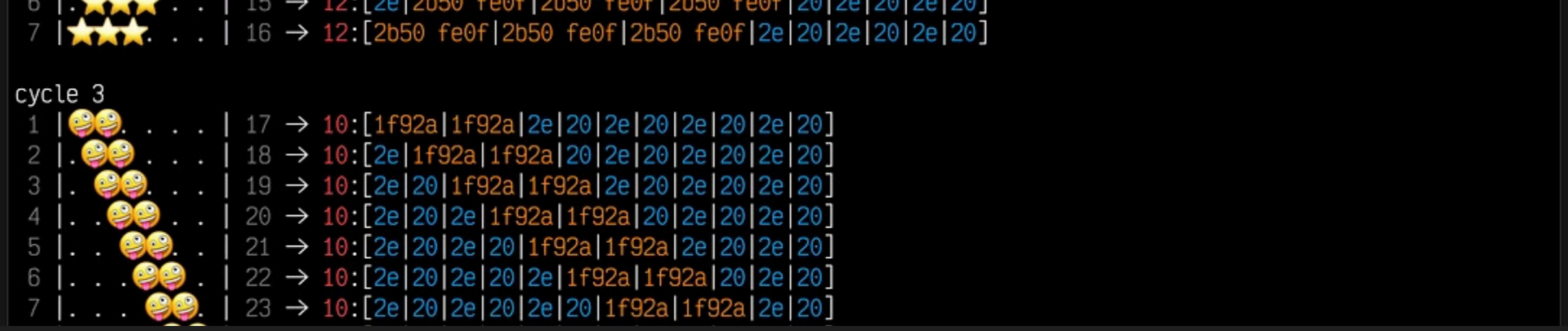
Third-Party Packages

These are community-developed libraries, not built into Python. They offer specialized tools for data science, web development, and more.

Importance in Data Science

`pip` is crucial for data science, allowing access to libraries like NumPy, Pandas, Scikit-learn, and Matplotlib for efficient data manipulation, analysis, and visualization.





Installing Packages with `pip`

Mastering package installation is crucial for Python developers, especially in data science, where numerous specialized libraries are used. `pip` streamlines this process, allowing you to easily extend Python's capabilities.

The `pip install` Command

This is the primary command to fetch and install packages from PyPI (Python Package Index). It handles dependencies automatically.

```
pip install package_name
```

Installing Key Data Science Libraries

You can install multiple packages at once by listing them. For example, to get `numpy`, `pandas`, and `matplotlib`:

```
pip install numpy pandas matplotlib
```

Verifying Installation

After installation, you can confirm a package is available by attempting to import it in a Python interpreter or using `pip show`.

```
python -c "import numpy;
print(numpy.__version__)"
```

Common Installation Errors

Watch out for permission issues (use `--user` or virtual environments), network problems, or missing build tools (e.g., C++ compilers for some packages).

Best Practices

Always use **virtual environments** (`venv`) to isolate project dependencies. Manage project requirements with a `requirements.txt` file (`pip freeze > requirements.txt`).

Made with GAMMA

Final Revision Notes: Key Takeaways

This presentation covered essential Python concepts for data science, focusing on tools and techniques to write efficient, reusable, and powerful code. Here's a quick summary of what we discussed:



Control Flow & Iteration

Mastered `for` and `while` loops for automating repetitive tasks and processing data sequences, along with control statements like `break` and `continue` for precise execution management.



Functions & Reusability

Explored how to define and use functions, including parameters, arguments, `*args`, `**kwargs`, and `return` statements. We also touched upon variable scope, `lambda` functions, and higher-order functions like `map()` for advanced programming.



Modules & Ecosystem

Understood the importance of modules for organizing code and extending Python's capabilities. We specifically examined the `random` module for generating randomness and `datetime` for handling time-series data.



Package Management with `pip`

Learned how `pip` simplifies installing, updating, and managing third-party libraries from PyPI, enabling access to powerful data science tools like NumPy, Pandas, and Matplotlib.

Made with GAMMA

Arithmetic:

- Performs basic arithmetic operations.

```python
# Addition
result = 2 + 3
```

Data types:

- Defines the type of values.

Strings:

- Manages and manipulates text data.

```python
s = "Python"
print(s.upper())
```

If else:

- Implements conditional logic.

```python
```

Variables:

- Stores values that can be reused and manipulated.

```python
age = 25
```

f-Strings:

- Formats strings using expressions inside curly braces.

```python
```

Python Data Science Cheat Sheet

Quickly reference essential Python concepts for your data science journey.

Control Flow

Master `for` and `while` loops for iteration. Use `break` and `continue` for precise flow management.

Functions

Define reusable blocks of code. Leverage `*args` and `**kwargs` for flexible input. Use `lambda` for anonymous functions.

Modules

Import modules (e.g., `random`, `datetime`) to extend Python's functionality with specialized tools.

Package Management

Use `pip install [package_name]` to add external libraries. Always work within virtual environments.

Made with GAMMA

Made with GAMMA